

1 StoryLens - Script Testing, Demo, Q&A

Người trình bày: **Nguyễn Lâm Phú Quý - QA & Business Analyst** Phạm vi: **Slide 20 -> 28** Bản này là bản Markdown để dễ chỉnh sửa trước khi chuyển sang PDF.

1.1 Cách dùng nhanh

- Phần chính cần nắm chắc nhất: **Slide 20 - Kiểm thử & CI/CD**.
 - Slide 21-24: nói theo góc nhìn QA/test cho tính năng và demo.
 - Slide 25-28: nói hạn chế, roadmap, kết luận nếu bạn là người chốt cuối.
 - Nếu bị hỏi sâu, dùng phần **Q&A vấn đáp** và **Glossary technical** bên dưới
 - Lưu ý quan trọng: PA5 report thật hiện ghi Katalon chạy Login/Navigation. Nếu slide vẫn ghi Upload/Reader, nói an toàn là: “Katalon kiểm thử UI tự động theo yêu cầu PA5; ngoài ra các luồng Upload/Reader/Q&A được bao phủ trong test plan, manual/UAT và các tầng test khác.”
-

2 Script trình bày

2.1 Slide 20 - Kiểm thử & CI/CD

Ở phần này, em xin trình bày cách mà nhóm kiểm soát chất lượng cho StoryLens.

Theo phân lý thuyết testing, thì kiểm thử có hai mục tiêu. (đó chính là **validation** và **defect testing**).

- Một là **validation**, tức là chứng minh hệ thống đáp ứng yêu cầu.
- Hai là **defect testing**, tức là cố gắng tìm lỗi khi hệ thống chạy sai đặc tả.

Với StoryLens, không chỉ là một website tĩnh. Mà có đầy đủ frontend, backend, AI Module, Supabase và AI Service. Vì vậy nếu chỉ bấm thử thủ công thì rất dễ bỏ sót lỗi. Nên nhóm chia kiểm thử thành nhiều tầng, mỗi tầng bắt một nhóm lỗi khác nhau.

Đầu tiên là **Katalon Studio**. Là tool kiểm thử giao diện tự động. Nói dễ hiểu là thay vì người test tự bấm từng bước, Katalon sẽ mở trình duyệt, nhập dữ liệu, click nút và kiểm tra kết quả theo script.

Trong StoryLens, nếu thầy hỏi cụ thể thì nhóm dùng Katalon cho các luồng UI ổn định như **Login** và **Navigation** trong PA5. Ví dụ: mở trang đăng nhập, nhập tài khoản sai, bấm login, rồi verify là hệ thống vẫn ở /login và có thông báo lỗi. Hoặc vào trang chủ, bấm qua Browse/Library để kiểm tra điều hướng đúng. Nhóm chọn các luồng này vì ít phụ thuộc AI bên ngoài, nên test tự động ổn định hơn.

Tiếp theo là **unit và component test ở frontend**. Unit test là kiểm thử một phần nhỏ, còn component test là kiểm thử một khối giao diện React. Nhóm dùng **Vitest** và **React Testing Library** cho các phần như rating, credit badge, reading list, local storage. Ví dụ nếu user bấm chọn rating thì UI phải cập nhật đúng; hoặc reading list phải lưu/đọc trạng thái đúng từ local storage. Các test này chạy nhanh, nên phù hợp để bắt lỗi hồi quy mỗi khi sửa UI.

Ở backend, nhóm dùng **pytest**. Đây là framework test phổ biến cho Python. Trong đồ án, pytest dùng để kiểm tra logic của backend FastAPI và các service phía server. Với những service có gọi Supabase hoặc Gemini, nhóm dùng **mock**. Mock nghĩa là giả lập phản hồi của dịch vụ ngoài, để kiểm tra logic backend mà không cần gọi API thật liên tục. Ví dụ thay vì thật sự gọi Gemini mỗi lần test, mình giả lập Gemini trả về một bản dịch mẫu, rồi kiểm tra backend xử lý kết quả đó đúng không. Cách này giúp test ổn định và không tốn quota Gemini.

Và Playwright cho E2E. E2E nghĩa là kiểm thử đầu-cuối trên trình duyệt, gần giống cách người dùng thật thao tác. Trong StoryLens, Playwright phù hợp cho các luồng như đăng nhập, trang chủ, protected pages, mobile, forum

và edge case. Ví dụ user chưa đăng nhập mà vào trang cần quyền thì phải bị chuyển về login; hoặc trên mobile thì menu và navigation vẫn phải dùng được.

Và trước khi push lên deploy thì sẽ có **kiểm thử tĩnh**. Kiểm thử tĩnh nghĩa là kiểm tra code mà chưa cần chạy app như người dùng. Ví dụ TypeScript typecheck bắt lỗi kiểu dữ liệu, ESLint bắt lỗi coding style và các pattern dễ gây bug, còn Python type hints giúp code backend rõ kiểu hơn.

Các tầng này được tự động hóa bằng **GitHub Actions CI**. Chia thành 5 job chính: typecheck/test, lint, production build, Playwright E2E và backend pytest.

[Quy tắc của nhóm là pull request phải được review và CI xanh trước khi merge vào main.]

-> Automation **không thay thế hoàn toàn manual testing**. Vì nó phù hợp nhất với các test lặp lại nhiều lần, test hồi quy, hoặc các luồng quan trọng. Còn các phần cần đánh giá bằng cảm nhận người dùng, ví dụ bản dịch có tự nhiên không, overlay có dễ đọc không, Studio QC có thuận tay không, thì nhóm vẫn cần manual test.

2.2 Slide 21 - Tính năng nổi bật

Đầu tiên là **Reader nâng cao**. Reader hỗ trợ đọc bản dịch dạng overlay, tức là chữ tiếng Việt được phủ trực tiếp lên bong bóng thoại. Và user có thể đọc song ngữ để đối chiếu hoặc cải thiện khả năng đọc và xem cảnh báo nếu độ tin cậy OCR thấp.

Studio QC. Là option để người dùng có thể chỉnh sửa lại kết quả dịch của AI. Có thể sửa văn bản, tìm-thay thế hàng loạt, và gửi phản hồi tốt/xấu. Bởi vì OCR và dịch tự động không thể cover hết các case được.

Chức năng Sharing. Sau khi dịch xong, người dùng có thể xuất bản chương công khai, cho phép người khác đọc ẩn danh, hoặc chia sẻ link có thời hạn.

Ngoài ra còn có forum để mọi người có thể bình luận chương dịch; phần **gamification** như bookmark, rating, reading list, mục tiêu đọc, XP và thành tựu; và phần **credit/thanh toán** để giới hạn lượt sử dụng cũng như là cũng có free credit mỗi ngày cho người dùng.

2.3 Slide 22 - Demo luồng A: Trang chủ & Upload

Đầu tiên người dùng sẽ thấy landing page chính. Sau đó người dùng đăng nhập, vào trang Upload, rồi cung cấp một ảnh cần dịch.

Người dùng chỉ việc upload ảnh lên còn lại pipeline sẽ xử lý: YOLOv8 phát hiện bong bóng, manga-ocr đọc chữ, LaMa xóa nền chữ gốc, rồi Gemini dịch sang tiếng Việt. Trong quá trình xử lý, giao diện hiển thị tiến trình để người dùng biết hệ thống đang chạy tới bước nào.

Ở đây các file sai định dạng hoặc quá kích thước phải bị chặn. Cũng như là nếu AI Service có vấn đề thì hệ thống thông báo lỗi thay vì crash.

2.4 Slide 23 - Demo luồng A: Reader overlay & Studio QC

Bản dịch tiếng Việt được phủ trực tiếp lên cũng như là có thể bật-tắt bản gốc để so sánh, xem song ngữ khi cần.

Nếu bản dịch hoặc OCR chưa đúng, người dùng chuyển sang **Studio QC**.

2.5 Slide 24 - Demo luồng B: Batch, RAG Q&A, Thư viện

Mở rộng sang cả chương và phần RAG.

Hệ thống xử lý tuần tự và hiển thị tiến trình thay vì thao tác từng trang một.

Tiếp theo là **RAG Q&A**. Người dùng có thể đặt câu hỏi về nội dung truyện, ví dụ hỏi về nhân vật, tính tiết hoặc một đoạn hội thoại. Hệ thống sẽ tìm các đoạn liên quan trong dữ liệu truyện, rồi dùng Gemini để trả lời kèm nguồn trích.

2.6 Slide 25 - Hạn chế & lỗi đã biết

Bên cạnh đó cũng còn các hạn chế nhất định:

Thứ nhất là **cold start**. Do hiện tại pipeline chỉ sử dụng các free-tier service như backend Render và AI Module trên HuggingFace Spaces, nên dịch vụ có thể ngủ sau một thời gian rảnh. Lần đầu truy cập có thể chậm khoảng 15 đến 60 giây. Nhóm giảm tác động bằng keep-alive và health check.

Thứ hai là **quota của Gemini**. Vì dùng free tier nên có giới hạn request mỗi ngày. Nhóm xử lý bằng xoay vòng key, throttle và cache, nhưng vào giờ cao điểm vẫn có thể bị giới hạn.

Thứ ba là **OCR với chữ hiệu ứng hoặc chữ viết tay**. Những kiểu chữ này khó hơn chữ in trong bong bóng thoại, nên độ chính xác có thể giảm.

Và **bố cục chữ Việt**. Tiếng Việt có dấu và thường dài hơn nên đôi khi phải thu nhỏ font hoặc cắt bớt.

Các lỗi đang xử lý gồm token phiên hết hạn giữa chừng, WebSocket mất gói khi mạng chập chờn, signed URL ảnh hết hạn trong phiên đọc dài, và hiệu năng nền anime trên thiết bị mobile cũ.

2.7 Slide 26 - Hướng phát triển

Về hướng phát triển sau này, nhóm có sáu hướng chính:

Cải thiện OCR và chất lượng dịch dịch và thêm các cặp ngôn ngữ như Hàn hoặc Anh.

Hai là cải thiện thứ tự đọc bong bóng phải-sang-trái cho các bố cục manga phức tạp.

Giảm cold start bằng cách chuyển sang tier trả phí và dùng hàng đợi tác vụ chuyên dụng.

Bốn là cá nhân hóa trải nghiệm đọc của người dùng và cải thiện RAG đa trang hoặc cả series.

Bổ sung công cụ kiểm duyệt và quy trình chuẩn bản quyền/DMCA rõ ràng hơn.

Sáu là đóng gói PWA thành ứng dụng di động để trải nghiệm đọc offline mượt hơn.

2.8 Slide 27 - Kết luận

Tóm lại, StoryLens là một sản phẩm hoàn chỉnh.

Hệ thống có ba dịch vụ triển khai độc lập: frontend, backend và AI Module. Mô hình phát hiện bong bóng đạt mAP@0.5 là 95,3%; OCR tiếng Nhật có CER 6,1%; và hơn chín nhóm yêu cầu chức năng đã được hiện thực.

Về technical, đã xây dựng được 1 pipeline AI end-to-end từ phát hiện, OCR, xóa nền, dịch ngữ cảnh đến RAG Q&A.

Về mặt công nghệ phần mềm, nhóm áp dụng RUP rút gọn kết hợp Scrum, có tài liệu use case, SAD, test plan, test case, test report, và CI để kiểm soát chất lượng.

2.9 Slide 28 - Cảm ơn & Q&A

Phần trình bày của nhóm em đến đây là hết. Nhóm em xin cảm ơn thầy và các bạn đã lắng nghe.

3 Bảng tra nhanh thuật ngữ testing

Bảng này **không cần trình bày trong slide**. Chỉ dùng để nhìn nhanh nếu thầy hỏi “cái X là gì?”, “vì sao dùng X?”, hoặc “trong đồ án em dùng ở đâu?”.

Thuật ngữ	Nói ngắn gọn là gì?
Katalon Studio	Công cụ kiểm thử UI tự động, có record/script/report.
Test automation	Dùng máy/công cụ để chạy test tự động, so sánh kết quả và report.
Manual testing	Người thật tự thao tác và đánh giá kết quả.
UAT	User Acceptance Testing: kiểm thử xem sản phẩm có chấp nhận được với yêu cầu/người dùng thật.
Validation testing	Kiểm thử để chứng minh hệ thống đáp ứng yêu cầu.
Defect testing	Kiểm thử để tìm lỗi bằng input sai/edge case.
Unit test	Kiểm thử một hàm/module nhỏ, cô lập.
Component test	Kiểm thử một component UI riêng lẻ.
Integration test	Kiểm thử nhiều module phối hợp với nhau.
System testing	Kiểm thử hệ thống đã tích hợp.
E2E test	End-to-end test: kiểm thử đầu-cuối như người dùng thật.
Regression test	Chạy lại test để chắc sửa cái mới không phá cái cũ.
Black-box testing	Test theo input/output, không cần biết code bên trong.
White-box testing	Test dựa trên hiểu biết code bên trong.
Requirements-based testing	Thiết kế test từ yêu cầu/use case.
Equivalence partitioning	Chia input thành nhóm tương đương để chọn đại diện test.
Boundary testing	Test giá trị biên.
Test case	Kịch bản test gồm bước, dữ liệu, expected result, actual result.
Test data	Dữ liệu dùng để chạy test.
Expected result	Kết quả mong đợi theo yêu cầu.
Actual result	Kết quả thực tế sau khi chạy test.
Test report	Tài liệu tổng hợp kết quả test.
Defect/Bug	Sai lệch giữa actual result và expected result.
Assertion/Checkpoint	Điều kiện kiểm tra Pass/Fail trong script tự động.
Record-playback	Ghi thao tác người dùng rồi phát lại.
Data-driven testing	Một script chạy với nhiều bộ dữ liệu.
Keyword-driven testing	Test từ các keyword/hành động mức cao.
Mock	Giả lập phản hồi của service ngoài.
Staging environment	Môi trường giống production nhưng dành riêng cho test.
Flaky test	Test lúc pass lúc fail do môi trường/dịch vụ ngoài, không hẳn do bug.
Test coverage	Mức độ yêu cầu/chức năng/code được test bao phủ.
CI	Continuous Integration: tự chạy kiểm tra khi có code mới.
CI xanh	Các job kiểm tra tự động đều pass.
Vitest	Framework test JavaScript/TypeScript nhanh.
React Testing Library	Test React theo hành vi người dùng nhìn thấy.
pytest	Framework test Python.
respx	Thư viện mock HTTP cho Python/httpx.
Playwright	Framework E2E browser automation.
ESLint	Công cụ lint JavaScript/TypeScript.
TypeScript typecheck	Kiểm tra kiểu dữ liệu mà không chạy app.
Python type hints	Chú thích kiểu dữ liệu trong Python.

4 Q&A vấn đáp có thể bị hỏi

4.1 1. Nhóm dùng những công cụ kiểm thử gì?

Nhóm dùng Katalon Studio cho kiểm thử UI tự động theo yêu cầu PA5; Vitest + React Testing Library cho unit/component test frontend; pytest + respx cho backend test và mock các API như Supabase/Gemini; Playwright cho E2E trình duyệt; TypeScript typecheck, ESLint, Python type hints cho kiểm thử tĩnh; và GitHub Actions để tự động hóa CI.

4.2 2. Vì sao nhóm chọn Katalon cho PA5?

Vì PA5 yêu cầu một công cụ test automation, có test case, test script và kết quả Passed/Failed. Katalon là công cụ phổ biến trong nhóm automation tools, hỗ trợ record-playback, script và report nên rất hợp để nộp minh chứng.

Với đồ án StoryLens, nhóm dùng Katalon để tự động hóa một số luồng UI ổn định; còn các luồng AI nặng như Upload -> dịch -> Reader vẫn được kiểm thử thêm bằng test plan, manual/UAT và các tầng test khác.

4.3 3. Test case lấy từ đâu ra?

Test case được trace từ use case và yêu cầu phần mềm. Đây là requirements-based testing: xem từng yêu cầu/use case rồi dẫn ra các bước, input, output và expected result.

Ví dụ UC01 Upload & Dịch trang, UC02 Reader overlay, UC04 Q&A RAG, cộng thêm các yêu cầu phi chức năng như bảo mật, hiệu năng, giới hạn file, phân quyền, và xử lý lỗi. PA4 có test plan, test cases và test report riêng để thể hiện mapping này.

4.4 4. Vì sao phải có nhiều tầng test?

Vì mỗi tầng bắt một loại lỗi khác nhau. Unit test bắt lỗi logic nhỏ; component test bắt lỗi UI component; backend test bắt lỗi nghiệp vụ/API; E2E test kiểm tra luồng người dùng; static check bắt lỗi kiểu dữ liệu và coding style.

Nếu chỉ test thủ công ở cuối thì rất dễ bỏ sót lỗi hồi quy.

4.5 5. Test automation có thay thế manual testing không?

Không. Theo bài Test Automation, automation không thay thế manual testing. Automation tốt cho các test lặp lại, regression test, high-risk/business-critical flow, hoặc test tốn thời gian.

Manual/UAT vẫn cần cho UI/UX và các đánh giá cần con người, ví dụ bản dịch đọc có tự nhiên không, overlay có dễ nhìn không, hoặc Studio QC có thuận tay không.

4.6 6. Katalon test gì trong PA5?

Câu trả lời an toàn:

Katalon được dùng để kiểm thử UI tự động theo rubric PA5: tối thiểu 2 use case, mỗi use case 2 scenario, có script và kết quả Passed/Failed.

Bản report PA5 hiện ghi các test chạy trên production thật gồm các luồng Login và Navigation. Các luồng Upload/Reader/Q&A được bao phủ thêm trong test plan PA4, manual/UAT, pytest, Vitest và Playwright.

4.7 7. Vì sao không dùng Katalon để test toàn bộ luồng Upload -> AI dịch -> Reader?

Vì luồng đó phụ thuộc nhiều dịch vụ ngoài: HuggingFace AI Module, Gemini quota, Supabase Storage và cold start. Nếu dùng trực tiếp trong Katalon để chấm tự động thì dễ flaky, tức là lúc pass lúc fail không hẳn do bug.

Nhóm vẫn kiểm thử luồng này bằng test case PA4, manual/UAT và các test backend/frontend; còn Katalon ưu tiên các luồng UI ổn định để chứng minh automation chạy thật.

4.8 8. Mock Supabase/Gemini có làm test kém thật không?

Mock không thay thế hoàn toàn integration test, nhưng rất cần cho unit/integration test ổn định.

Nó giúp kiểm tra logic xử lý của hệ thống mà không phụ thuộc mạng, quota hoặc dữ liệu production. Với các luồng thật, nhóm bổ sung manual/UAT và demo production.

4.9 9. Playwright khác Katalon ở đâu?

Katalon là công cụ test automation low-code/record-script, phù hợp báo cáo PA5 vì có test script và report rõ.

Playwright là framework E2E bằng code, phù hợp tích hợp CI, chạy headless, test nhiều trình duyệt/kích thước màn hình và kiểm thử regression lâu dài.

4.10 10. CI có những job nào?

CI có 5 job chính: typecheck/test frontend, lint, production build, Playwright E2E và backend pytest.

Mục tiêu là mọi pull request phải qua kiểm tra tự động và review trước khi merge vào main.

4.11 11. Nếu test pass 100% thì có nghĩa là hệ thống hết lỗi chưa?

Không. Theo lý thuyết testing, test chỉ chứng minh sự hiện diện của lỗi, không chứng minh hệ thống hoàn toàn không còn lỗi.

Test pass nghĩa là hệ thống đạt các kịch bản đã kiểm thử. Vẫn có rủi ro ở các ca chưa bao phủ như tải cao, mạng chập chờn, quota Gemini, dữ liệu manga lạ, thiết bị mobile cũ.

4.12 12. Validation testing khác defect testing như thế nào?

Validation testing là kiểm thử để chứng minh phần mềm đáp ứng yêu cầu. Ví dụ upload ảnh hợp lệ thì hệ thống xử lý và hiển thị bản dịch.

Defect testing là kiểm thử để tìm lỗi. Ví dụ upload file sai định dạng, link hết hạn, token hết hạn, hoặc user không có quyền truy cập dữ liệu của người khác.

Trong StoryLens, nhóm có cả hai: test luồng đúng và test luồng lỗi/edge case.

4.13 13. Hạn chế lớn nhất của phần testing hiện tại là gì?

Hạn chế lớn nhất là chưa có staging environment riêng với Supabase/Gemini test credentials để chạy full E2E cho luồng Upload -> AI dịch -> Reader -> Q&A một cách ổn định mà không đụng production.

Đây là hướng cải thiện quan trọng sau PA5.

5 Vì sao dùng Y mà không dùng X?

5.1 Vì sao dùng Katalon mà không dùng Selenium?

Katalon phù hợp với yêu cầu PA5 hơn vì nó cho ra test case, test script và report Passed/Failed rất rõ, dễ nộp như một tài liệu kiểm thử tự động.

Selenium mạnh và phổ biến, nhưng nếu dùng trực tiếp thì nhóm phải tự viết nhiều phần khung báo cáo hơn. Với mục tiêu PA5 là chứng minh automation UI cho một số use case, Katalon nhanh, đủ rõ ràng và dễ tái hiện.

5.2 Vì sao vẫn dùng Playwright nếu đã có Katalon?

Katalon phục vụ tốt cho báo cáo automation PA5, còn Playwright phù hợp hơn cho regression test lâu dài trong CI. Playwright viết bằng code, chạy headless tốt, dễ tích hợp GitHub Actions, dễ test mobile viewport và nhiều trạng thái trình duyệt.

Nói ngắn gọn: Katalon để trình bày automation rõ ràng; Playwright để bảo vệ codebase lâu dài.

5.3 Vì sao dùng Vitest mà không dùng Jest?

Frontend của nhóm dùng Next.js/React hiện đại và tooling JavaScript mới, nên Vitest chạy nhanh, cấu hình nhẹ, tích hợp tốt với jsdom.

Jest vẫn dùng được, nhưng Vitest phù hợp hơn với nhu cầu test unit/component nhanh trong repo hiện tại.

5.4 Vì sao dùng React Testing Library mà không test trực tiếp state/component internals?

React Testing Library khuyến khích test theo hành vi người dùng nhìn thấy: text nào xuất hiện, button nào bấm được, form phản hồi thế nào.

Cách này bền hơn khi refactor component, vì test không phụ thuộc quá sâu vào implementation detail bên trong.

5.5 Vì sao dùng pytest mà không dùng unittest mặc định của Python?

unittest là thư viện chuẩn và vẫn tốt, nhưng pytest viết test gọn hơn, fixture mạnh hơn, output dễ đọc hơn và hệ sinh thái plugin phong phú.

Backend FastAPI/Python của nhóm có nhiều service cần mock, fixture và async test, nên pytest thuận tiện hơn.

5.6 Vì sao dùng respx mà không gọi Gemini/Supabase thật trong mọi test?

Nếu test nào cũng gọi API thật thì kết quả dễ phụ thuộc mạng, quota, trạng thái dịch vụ ngoài và dữ liệu production. respx giúp mock HTTP response để kiểm tra logic backend ổn định. Nhóm vẫn demo và UAT trên môi trường thật, nhưng unit/integration test nên cô lập để chạy nhanh và đáng tin cậy.

5.7 Vì sao không làm load test/JMeter?

Load test phù hợp khi đánh giá hệ thống nhiều người dùng đồng thời. Trong phạm vi PA4/PA5, trọng tâm là đúng chức năng, automation test và demo sản phẩm.

Hơn nữa AI Module đang chạy free tier, nếu load test mạnh có thể tốn quota hoặc làm dịch vụ bị giới hạn. Nhóm ghi nhận load/performance test là hướng mở rộng khi có staging riêng.

6 Glossary technical gắn với StoryLens

Thuật ngữ	Là gì?	Trong StoryLens dùng ở đâu?
Vercel	Nền tảng deploy frontend, tối ưu cho Next.js, có CDN.	Host frontend StoryLens: vercel.com
Render	Nền tảng deploy backend/API.	Host Backend API FastAPI
HuggingFace Spaces	Nền tảng host app/model AI.	Host AI Module gồm YOLOv8, manga-ocr
Supabase	Backend-as-a-Service dựa trên Postgres, có Auth, Storage, RLS.	Dùng làm database + auth
Docker	Đóng gói app và dependency vào container.	Backend API và AI Module
FastAPI	Framework Python để xây API.	Backend dùng cho auth, upload
Next.js	Framework React cho web app.	Frontend dùng Next.js 16
React	Thư viện xây UI bằng component.	Reader overlay, upload form
TypeScript	JavaScript có kiểu dữ liệu tĩnh.	Giảm lỗi sai kiểu props/API
YOLOv8	Model object detection.	Phát hiện bong bóng thoại
manga-ocr	Model OCR chuyên đọc chữ manga/tiếng Nhật.	Đọc chữ Nhật trong bounding box
LaMa inpaint	Kỹ thuật xóa vùng chữ và lấp nền ảnh.	Xóa chữ gốc trong bong bóng
Gemini 2.5 Flash	LLM của Google.	Dịch có ngữ cảnh và sinh lời
Embedding	Biểu diễn văn bản thành vector số.	Vector hóa câu hỏi và đoạn trả lời
RAG	Tìm tài liệu liên quan trước rồi đưa cho LLM trả lời.	Q&A tìm đoạn thoại liên quan
WebSocket	Kết nối realtime hai chiều.	Cập nhật tiến trình upload
Polling	Client hỏi server định kỳ.	Fallback khi WebSocket mất kết nối
Cookie HttpOnly	Cookie không đọc được bằng JavaScript.	Giữ token đăng nhập để ghi nhớ
RBAC	Phân quyền theo vai trò.	User/admin ở API.
RLS	Bảo mật từng dòng dữ liệu trong database.	User chỉ đọc/sửa dữ liệu của mình
Rate-limit	Giới hạn số request.	SlowAPI chống spam, brute force
Sentry	Theo dõi lỗi runtime trên production.	Ghi nhận lỗi frontend/backend
OpenTelemetry	Thu thập trace/metric/log.	Theo dõi latency/request graph
CI/CD	Tự chạy test/build và triển khai khi có code mới.	GitHub Actions chạy test/build
Unit test	Kiểm thử hàm/module nhỏ độc lập.	Backend test idempotency
Component test	Kiểm thử component UI riêng lẻ.	StarRating, Toast, CreditButton
Integration test	Kiểm thử nhiều module phối hợp.	Backend service phối hợp
System testing	Kiểm thử hệ thống đã tích hợp.	Upload -> pipeline -> Reader
E2E test	Kiểm thử đầu-cuối như người dùng thật.	Playwright test auth, home
Manual test	Kiểm thử thủ công bởi người thật.	Luồng AI phụ thuộc Gemini
UAT	User Acceptance Testing.	Kiểm thử sản phẩm có chấp nhận
Regression test	Chạy lại test để không phá chức năng cũ.	CI và test suite chống lỗi hời hợt
Validation testing	Chứng minh phần mềm đáp ứng yêu cầu.	Upload hợp lệ -> pipeline
Defect testing	Cố tìm lỗi bằng tình huống sai/edge case.	File sai định dạng, token hết hạn
Black-box testing	Test dựa trên input/output, không cần biết code.	Katalon/Playwright/manual
White-box testing	Test dựa trên hiểu biết code.	Unit test service cụ thể như auth
Requirements-based testing	Thiết kế test từ yêu cầu.	Test case PA4 trace từ UC
Equivalence partitioning	Chia input thành nhóm tương đương để chọn đại diện test.	Upload: file hợp lệ, sai MIME type
Boundary testing	Test giá trị biên.	Ảnh 10MB, batch 5-100 tr
Test case	Kịch bản test gồm steps, data, expected/actual result.	PA4 có test cases; PA5 có test data
Test data	Dữ liệu dùng để test.	Email/password, file ảnh n
Expected result	Kết quả mong đợi.	Login sai ở lại /login, file
Actual result	Kết quả thực tế khi chạy test.	Ghi trong test report để kế
Test report	Tài liệu tổng hợp kết quả test.	PA4 có test_report.docx; P
Defect/bug	Sai lệch giữa actual result và expected result.	Nếu fail thì ghi bước tái hi
Record-playback	Ghi thao tác người dùng rồi phát lại.	Katalon hỗ trợ; cần checkp
Assertion/checkpoint	Điều kiện kiểm tra Pass/Fail	Verify URL vẫn là /login

